

tf.compat.v1.image.extract_patches Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/image/extract_patches

Extract patches from images.

Function Signatures

```
tf.compat.v1.image.extract_patches( images, sizes, strides,
rates, padding, name=None )
```

Code Examples

```
tf.compat.v1.image.extract_patches(
images, sizes, strides, rates, padding, name=None
)
```

```
tf.compat.v1.image.extract_patches(
images, sizes, strides, rates, padding, name=None
)
```

```

n = 10
# images is a 1 x 10 x 10 x 1 array that contains the numbers 1
through 100
images = [[[[x * n + y + 1] for y in range(n)] for x in range(n)]]
# We generate two outputs as follows:
# 1. 3x3 patches with stride length 5
# 2. Same as above, but the rate is increased to 2
tf.image.extract_patches(images=images,
    sizes=[1, 3, 3, 1],
    strides=[1, 5, 5, 1],
    rates=[1, 1, 1, 1],
    padding='VALID')
# Yields:
[[[ [ 1  2  3 11 12 13 21 22 23]
    [ 6  7  8 16 17 18 26 27 28]]
  [[51 52 53 61 62 63 71 72 73]
    [56 57 58 66 67 68 76 77 78]]]]

```

```

n = 10
# images is a 1 x 10 x 10 x 1 array that contains the numbers 1
through 100
images = [[[[x * n + y + 1] for y in range(n)] for x in range(n)]]
# We generate two outputs as follows:
# 1. 3x3 patches with stride length 5
# 2. Same as above, but the rate is increased to 2
tf.image.extract_patches(images=images,
    sizes=[1, 3, 3, 1],
    strides=[1, 5, 5, 1],
    rates=[1, 1, 1, 1],
    padding='VALID')
# Yields:
[[[ [ 1  2  3 11 12 13 21 22 23]
    [ 6  7  8 16 17 18 26 27 28]]
  [[51 52 53 61 62 63 71 72 73]
    [56 57 58 66 67 68 76 77 78]]]]

```

```

*   *   *   4   5   *   *   *   9 10
*   *   *  14 15   *   *   *  19 20
*   *   *  24 25   *   *   *  29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
*   *   *  54 55   *   *   *  59 60
*   *   *  64 65   *   *   *  69 70
*   *   *  74 75   *   *   *  79 80
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100

```

```

*   *   *   4   5   *   *   *   9 10
*   *   *  14 15   *   *   *  19 20
*   *   *  24 25   *   *   *  29 30
31 32 33 34 35 36 37 38 39 40
41 42 43 44 45 46 47 48 49 50
*   *   *  54 55   *   *   *  59 60
*   *   *  64 65   *   *   *  69 70
*   *   *  74 75   *   *   *  79 80
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100

```

```

tf.image.extract_patches(images=images,
    sizes=[1, 3, 3, 1],
    strides=[1, 5, 5, 1],
    rates=[1, 2, 2, 1],
    padding='VALID')
# Yields:
[[[ [ 1   3   5  21  23  25  41  43  45]
    [ 6   8  10  26  28  30  46  48  50]]
  [[ 51  53  55  71  73  75  91  93  95]
    [ 56  58  60  76  78  80  96  98 100]]]]

```

```
tf.image.extract_patches(images=images,
                        sizes=[1, 3, 3, 1],
                        strides=[1, 5, 5, 1],
                        rates=[1, 2, 2, 1],
                        padding='VALID')
# Yields:
[[[ [ 1  3  5 21 23 25 41 43 45]
    [ 6  8 10 26 28 30 46 48 50]]
  [ [51 53 55 71 73 75 91 93 95]
    [56 58 60 76 78 80 96 98 100]]]]]
```

Documentation

Extract patches from images.

This op collects patches from the input image, as if applying a convolution. All extracted patches are stacked in the depth (last) dimension of the output.

Specifically, the op extracts patches of shape sizes which are strides apart in the input image. The output is subsampled using the rates argument, in the same manner as "atrous" or "dilated" convolutions.

The result is a 4D tensor which is indexed by batch, row, and column. `output[i, x, y]` contains a flattened patch of size `sizes[1]`, `sizes[2]` which is taken from the input starting at `images[i, x*strides[1], y*strides[2]]`.

Each output patch can be reshaped to `sizes[1]`, `sizes[2]`, `depth`, where `depth` is `images.shape[3]`.

The output elements are taken from the input at intervals given by the rate

argument, as in dilated convolutions.

The padding argument has no effect on the size of each patch, it determines how many patches are extracted. If `VALID`, only patches which are fully contained in the input image are included. If `SAME`, all patches whose starting point is inside the input are included, and areas outside the input default to zero.

Example:

If we mark the pixels in the input image which are taken for the output with `*`, we see the pattern:

We can again draw the effect, this time using the symbols `*`, `x`, `+` and `o` to distinguish the patches:

Returns